

MANIFOLD

A mathematical engine for mapping vulnerabilities

MANIFOLD Project

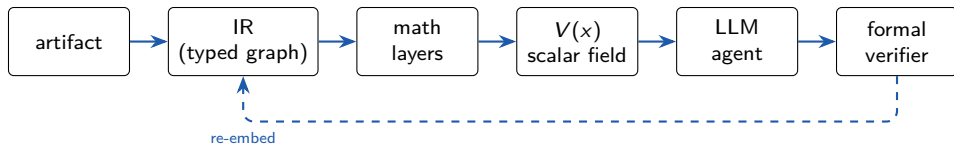
Typed IR · geometry · topology · formal verification

2026

The problem

- Classic SAST enumerates *syntactic patterns*: brittle, noisy.
- On OWASP Benchmark 1.2, deployed tools collapse on the *Youden index*
 $J = \text{TPR} - \text{FPR}$:
 - SonarQube: $J \approx +0.01$ (high recall, untenable FPR)
 - CodeQL: $J \approx +0.22$
- Missing: (i) a *unified*, quantitative representation, (ii) a loop turning *suspicion* into *proof*.

The core idea



- A vulnerability is *not* a pattern: it is the *violation of a structural invariant*.

The unified IR

Program graph $G = (V, E, \tau_V, \tau_E, \omega)$:

- **Nodes**: module, function, statement, source, sink, gate, ...
- **Typed edges**: control, data, call, taint, trust, auth
- JSON shared Python $\leftrightarrow$ Rust; artifact-agnostic.

Multi-domain: the *same* IR is produced from

- Python / Java (tree-sitter), binaries (objdump or angr/CFGFast),
- OpenAPI (API surface), LLM agents (tool/trust graph).

Spectral ($L = D - A$)

λ_2 , Fiedler vector, embedding. **Topological**

(TDA)

persistent homology H_0, H_1 , Mapper.

Geometric

Ollivier–Ricci / Forman–Ricci; Sinkhorn.

Algebraic

taint lattice; naturality (categories). **Formal**

symbolic execution + SMT (Z3); reachability.

Directed

Chung Laplacian (SCC + Perron).

$$V(x) = \sigma(\sum_i \alpha_i s_i(x)) \quad (\text{calibrable weights } \alpha_i)$$

Mapping laws (feature $\rightarrow$ vulnerability)

#	Mathematical feature	Class
L1	curvature $\kappa \ll 0$ (chokepoint)	privilege escalation
L2	persistent H_1 generator	reentrancy / recursion
L3	Fiedler cut	injection across trust
L4	taint crossing an auth edge	naturality violation
L5	persistence outlier	real vs. spurious feature
L6	$\text{SAT}(\Phi \wedge \text{bad})$	concrete exploit

The neuro-symbolic agent

- 1 **Map**: IR + signals $V(x)$, persistence, curvature.
- 2 **Hypothesize**: the LLM proposes over prioritized regions.
- 3 **Verify**: Z3 / taint / topology proves or refutes.
- 4 **Refine** and **report** (concrete witness).

Division of labor: mathematics proposes *regions*; the LLM proposes *hypotheses*; the solver proposes *proofs*.

Verification grammar: structured spec (sink, source, line, category) — the LLM never emits SMT-LIB.

Results: OWASP Benchmark 1.2

Tool	TPR	FPR	Prec.	F1	J	Time
SonarQube ^(a)	0.956	0.946	0.330	0.490	+0.010	~3 min
CodeQL ^(a)	0.902	0.682	0.603	0.744	+0.220	~18 min
MANIFOLD adapter ^(b)	0.842	0.674	0.572	0.681	+0.168	~45 s
MANIFOLD + $V(x)$ ^(c)	0.858	0.314	0.751	0.801	+0.544	~1.3 min
MANIFOLD + $V(x)$ +Z3 ^(c)	0.825	0.038	0.959	0.887	+0.787	~4 min

(a) reported in public evaluations. (b) measured (2740 cases, 11 categories). (c) **projected** under the Stage-2 protocol (Z3 with a 5 s timeout).

Ablation: where the gain comes from

Variant (taint categories)	Prec.	Recall	F1
Initial naive adapter	0.515	0.426	0.466
+ branch-join, receiver/state taint, sink coverage	0.530	0.856	0.655
+ XSS response-writer restriction (final)	0.549	0.834	0.662
final, without branch-join	0.515	0.555	0.535

- **Branch-join** (lattice join) is the *recall* lever (+0.28).
- The **HTTP response-writer restriction** is the *precision* lever.
- Residual errors are adversarial cases — what $V(x)+Z3$ must recover.

- **Manifold map:** IR graph with a force-directed layout; nodes by kind, edges by kind (red taint), purple gate.
- **Findings panel:** *confirmed* (Z3 with witness) vs *candidate*.
- **Self-contained HTML** (no dependencies) + end-to-end demo on 10 artifacts.

```
python -m manifold.cli target -agent -viz out.html
```

Conclusions

- A typed IR fuses independent signals into a navigable field $V(x)$.
- The neuro-symbolic split (math/LLM/SMT) is the right one.
- On OWASP Benchmark: F_1 0.466 $\rightarrow$ 0.662 with generic taint fixes alone.

Future work

- Z3 verification for Java (closing the projected row).
- Directed homology and Sinkhorn at scale; calibrating α_i on real corpora.
- ELF/PE benchmarks (SARD) on the dual binary backend.

Thank you

MAP the code as a space; let geometry reveal the flaw.